Important additional instruction for Phase 1:

The files currently located in app/data/corpus/ are the original documents supplied by the enterprise. Treat them as the authoritative source.

Do not generate, rewrite, modify, normalize, or replace the original corpus files.

Before implementing the loader or chunker, inspect the actual Markdown structure of all documents, including headings, subsections, lists, tables, and unusually long sections.

Base the chunking strategy on the actual corpus, not assumptions.

The ingestion pipeline must preserve the original source content and enough metadata to trace every generated chunk back to its exact source document and section.

Keep generated chunks in memory or in a separate temporary/generated-data location. Never overwrite the original corpus.

First present your analysis and proposed chunking strategy. Wait for approval before implementing.

PHASE 1 — Document Ingestion and Chunking

Objective

Implement the document ingestion subsystem that loads local pharmacology documents and transforms them into structured text chunks suitable for embedding and retrieval.

Do not implement embeddings or vector search yet.

Step 1 — Inspect actual documents

Inspect the corpus before implementing.

Determine:

- Supported extensions.
- Encoding variations.
- Document organization.
- Markdown heading patterns.
- Section structures.
- Potentially empty or malformed documents.

Do not assume all documents have identical structures.

Step 2 — Implement document loading

Create a loader responsible for:

- Discovering `.md` and `.txt` files inside the configured corpus directory.
- Reading files safely.
- Handling encoding issues explicitly.
- Skipping or reporting invalid files appropriately.
- Preserving original filenames.
- Returning structured document objects.

Do not read arbitrary filesystem paths supplied by users.

Step 3 — Implement chunking

Implement a chunking strategy appropriate for pharmacological documents.

Prefer section-aware chunking.

Requirements:

- Preserve section headings.
- Avoid splitting sentences unnecessarily.
- Avoid producing empty chunks.
- Support configurable chunk size and overlap.
- Preserve source filename.
- Preserve section metadata.
- Assign stable chunk identifiers where practical.

Use an initial chunk-size estimate of approximately 400–700 tokens and overlap of 50–100 tokens only as a starting point.

If a tokenization library is needed, justify the choice. Do not pretend that character counts are token counts.

Step 4 — Data structures

Define clear structures for:

- Loaded documents.
- Document chunks.
- Source metadata.

Use Pydantic models or dataclasses where appropriate.

Avoid unnecessary abstractions.

Step 5 — Tests

Create tests covering:

- Markdown loading.
- Text loading.
- Multiple documents.
- Empty files.
- Unsupported extensions.
- Encoding errors.
- Heading preservation.
- Chunk boundaries.
- Overlap behavior.
- Metadata preservation.
- Missing corpus directory.

Use temporary test files rather than depending exclusively on the real corpus.

Constraints

- No embeddings.
- No vector database.
- No LLM calls.
- No agent orchestration.
- No frontend implementation.
- Do not modify original corpus documents.

Deliverables

Provide:

1. Architecture explanation.
2. Files created or modified.
3. Chunking strategy and rationale.
4. Example input and output chunks.
5. Test results.
6. Known limitations.

Stop after Phase 1.